

MANUAL26. УРОК 3. ЖИЗНЕННЫЙ ЦИКЛ ПО: ГДЕ И ЗАЧЕМ ТЕСТИРОВАНИЕ В SDLC.

На прошлом уроке вы работали с моделью «ожидание -> факт -> риск». Это была точка зрения одной проверки. В этой теме мы поднимаемся уровнем выше и смотрим на весь путь продукта: от идеи до поддержки после релиза.

Что такое SDLC и зачем это знать тестировщику

SDLC (Software Development Life Cycle)

это жизненный цикл разработки продукта. Проще говоря, это карта этапов, через которые проходит любая функция:

сначала ее придумывают, потом описывают, проектируют, реализуют, проверяют, выпускают и поддерживают.

Почему это важно именно для QA: качество не появляется в конце «магическим образом». Оно собирается постепенно. Если команда думает о рисках только перед релизом, обычно уже поздно, потому что часть ошибок пришла из требований или дизайна. Поэтому задача QA — работать не в одной точке, а на всей дистанции.

Этап 1. Идея и требования

На этом этапе команда определяет, какую пользовательскую проблему решает функция и какой результат считается правильным. Здесь часто рождаются самые дорогие ошибки: формулировка вроде «должно быть удобно» звучит красиво, но ее невозможно нормально проверить.

QA на этапе требований помогает сделать текст проверяемым. Не «быстро загружается», а «загружается за N секунд в

таких-то условиях». Не «работает стабильно», а «не падает при таких-то действиях». Когда критерии ясные, команда спорит меньше, а проверяет точнее.

Этап 2. Проектирование

Здесь команда решает, как именно функция будет устроена: какие состояния у интерфейса, как обрабатываются ошибки, что происходит при сбоях сети и что делать с несовместимыми данными. Это этап, где важно не только «как работает в идеале», но и «что будет, если что-то пойдет не так».

Роль QA — задавать неудобные, но полезные вопросы заранее. Что увидит пользователь при обрыве соединения? Что будет, если данные частично повреждены? Можно ли безопасно повторить действие? Чем лучше продуман дизайн в негативных сценариях, тем меньше аварий после релиза.

Этап 3. Разработка

На этапе разработки появляется код, конфигурации и новые

сборки. Для QA это не пауза, а рабочая фаза: нужно синхронизироваться с изменениями, обновлять тестовые сценарии и заранее готовить данные для проверки.

Частая проблема здесь — отрыв реализации от первоначальной цели. Фича может быть «технически готова», но не соответствовать тому, что договорились сделать для пользователя. Поэтому QA проверяет не только факт работы кода, но и соответствие требованиям.

Этап 4. Тестирование

На этом этапе гипотезы превращаются в факты. Проверяется логика, устойчивость, регрессия, поведение в разных условиях. Важно помнить: сильное тестирование — это не список из сотни формальных шагов, а фокус на рисках.

Главный результат QA здесь — не просто «нашли баг», а понятная информация для решения: условия, шаги, ожидание, факт, повторяемость и риск. Такой формат экономит время всей команды, потому что

дефект легче воспроизвести и исправить.

Этап 5. Релиз

Релиз — это точка принятия решения. QA не «ставит печать разрешено», а дает прозрачную картину: что проверено, что не проверено, какие риски приняты, какие дефекты критичны. Это позволяет выпускать осознанно, а не «на удачу».

Если в релизной картине нет ясности, команда обычно платит за это после выхода: горячими фиксами, срочными откатами и потерей доверия пользователей.

Этап 6. Поддержка и улучшения

После релиза приходит реальная жизнь: инциденты, обращения пользователей, странные кейсы, которые не видели в тестовой среде. Здесь очень важно не просто «чинить», а извлекать урок.

Каждый инцидент нужно разбирать вопросом: «на каком этапе это можно было поймать раньше?» Такой разбор делает

SDLC сильнее от версии к версии. Иначе команда будет бесконечно лечить симптомы, не меняя причину.

Как этапы связаны между собой

SDLC — это не набор отдельных коробок, а одна цепочка. Ошибка в требованиях почти всегда переезжает в дизайн, потом в код, потом в тестирование и в итоге доходит до пользователя. Поэтому ранняя QA-проверка всегда дешевле поздней.

Есть простое правило: каждый следующий этап должен давать четкий ответ на вопрос предыдущего. Если ответа нет, проблема переносится дальше и становится дороже.

Исключения на этапах: когда бывают и почему это не норма

В реальной работе бывают аварийные ситуации: например, критичный инцидент после релиза или срочный security-фикс. В такие моменты часть формальных действий

действительно может
сокращаться по времени.

Но это важно зафиксировать:
аварийный режим —
исключение, а не стиль работы.
Если команда постоянно
пропускает этапы «ради
скорости», качество быстро
разрушается. Появляются
повторные дефекты, релизы
становятся нервными, а
технический долг растет.

Допустимое исключение — это
когда риск уже высокий и нужно
быстро стабилизировать продукт.
Неприемлемая практика — когда
требования пропускают «чтобы
быстрее начать», релизят без
минимальной проверки
критичного пути или не делают
пост-разбор после срочного
фикса.

Даже в срочной задаче должны
остаться три опоры: понятный
контекст изменения, быстрая
проверка критичного сценария и
фиксация остаточного риска.
После стабилизации команда
возвращается к полному циклу
SDLC.

Сквозной пример: Minecraft

Представим ситуацию: команда готовит обновление Minecraft и меняет механизм сохранения мира и загрузки чанков. Цель благородная — ускорить загрузку и уменьшить задержки. Но одновременно растёт риск повредить старые сохранения.

На этапе требований QA вместе с командой уточняет, что считается «без потери данных»: сохраняются ли постройки, инвентарь, координаты игрока, состояния редстоуна и другие критичные элементы мира.

На этапе дизайна обсуждается схема миграции старых данных. QA поднимает риски: прерванная запись, нехватка места, конфликт версии клиента и сервера, поврежденные файлы сохранения. Если это не учесть заранее, потом придется спасать пользователей патчами.

На разработке QA готовит реалистичные наборы данных: старые миры разных версий, крупные миры, миры после долгих сессий. Это важно, потому что «идеальный тестовый мир» редко показывает реальные проблемы.

На тестировании проверяется не только счастливый сценарий, где все открывается, но и сбойные варианты: повторное открытие,

восстановление после прерывания, поведение на разных платформах, регрессия автосохранения и загрузки координат.

Перед релизом QA фиксирует рисковую картину: что уже покрыто, где есть ограничения, что обязательно мониторить в первые часы после выхода. Если есть риск потери прогресса, это уже повод обсуждать перенос релиза.

После релиза может выясниться, что у части игроков появляются «битые» чанки. Тогда задача QA — быстро собрать воспроизводимые кейсы, выделить общий паттерн, помочь команде выпустить патч и добавить сценарий в постоянную регрессию. Иначе проблема вернется в следующем обновлении.

Смысл примера простой: одну и ту же проблему можно поймать в разных точках цикла. Чем раньше ее ловят, тем меньше потери у игрока и команды.

Итоги занятия

Если вы можете посмотреть на дефект и назвать не только «что сломалось», но и «на каком

этапе это стоило поймать»,
значит вы мыслите как QA
системно. Это и есть цель темы
про SDLC.

Критерии успеха:

- Вы уверенно объясняете этапы SDLC простым языком.
- Для каждого этапа можете назвать роль QA и типичный риск.
- Умеее связать пользовательский сбой с этапом, где его можно было поймать раньше.
- Понимаете, почему режим «QA только в конце» приводит к дорогим ошибкам.

Глоссарий к занятию

1. **SDLC (Software Development Life Cycle)** — Жизненный цикл разработки продукта: от идеи до поддержки.
2. **Acceptance Criteria (критерии приемки)** — Проверяемые условия, при которых задача считается выполненной корректно.
3. **Release (релиз)** — Выпуск версии продукта для пользователей.
4. **Regression Testing (регрессионное тестирование)** — Проверка,

не сломались ли ранее
рабочие функции после
изменений.

5. **Risk (риск)** — Вероятность
нежелательного события и
масштаб его последствий.

6. **Incident (инцидент)** —
Проблема в рабочей среде,
которая затрагивает
пользователей и требует
реакции команды.